DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA0603 – Design and Analysis of Algorithms

Assignment: Brute Force Algorithms — Selection Sort & Sequential Search

Student Name	D. HUSSAINAIAH
Register Number	192425300
Department	Computer Science and Engineering
Programme	B.E./B.Tech. – Computer Science and Engineering
Course Code & Name	CSA0603 – Design and Analysis of Algorithms
Academic Year / Batch	2026 – Regular
Faculty Name	Dr. P. Solainayagi
Assignment Title	Analytical Study and Implementation of Brute Force Algorithms for Selection Sort and Sequential Search
Date of Issue	01-09-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome (CO)	CO2: Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull.
Bloom's Taxonomy Level	L4 – Analyse; L5 – Evaluate; L6 – Create

1. Assignment Problem / Challenge

Given the set of 12 numbers A = {42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21}, the Brute Force technique is applied to solve a sorting problem and a searching problem. For sorting, Selection Sort is used: on each pass the smallest element of the unsorted portion of the array is identified by a linear scan and placed into its correct position at the front of the array. For searching, Sequential Search is used to locate the key value 68 by examining the elements one by one, in order, from the beginning of the list. The sections below present the algorithms, a full step-by-step trace on the given data, the resulting sorted sequence and search outcome, the comparison/swap counts, and an analysis comparing the two brute-force approaches with more efficient alternatives.

2. Problem Statement

For Selection Sort, the position of the minimum element must be identified on every pass, the comparisons performed must be shown, and a correct algorithm/pseudocode must be developed. The total number of comparisons is derived analytically from the summation $(n - 1) + (n - 2) + \ldots + 1 = n(n - 1)/2$, and the number of swaps actually performed on the given input is counted and distinguished from the comparison count. For Sequential Search, the number of comparisons needed to locate the key 68 is determined, and the best-case, average-case and worst-case behaviour is discussed. The time complexity of both algorithms is expressed using Big-O, Big-Ω and Big-Θ notation, their behaviour as the input size n grows is examined, and the situations in which their brute-force nature becomes inefficient are identified. Finally, efficient alternatives — Merge Sort / Quick Sort for sorting and Binary Search for searching — are proposed, along with the conditions required for their use.

3. Requirements and Constraints

- A publicly available real-world numeric dataset is used — the "10,000 Sales Records" dataset (GitHub: azhar2ds/DataSets) — and subsets of it are drawn to test input sizes from 10^3 up to 10^6 records.
- Selection Sort (Brute Force), Merge Sort (Divide-and-Conquer) and, for searching, Sequential Search and Binary Search are implemented in the same language (Python 3) and run in the same environment for a fair comparison, on the same data subset per input size.
- Execution time and operation (comparison) counts are recorded for each input size, and the measured trend is related back to the theoretical $\Theta(n^2)$, $\Theta(n \log n)$, $\Theta(n)$ and $\Theta(\log n)$ complexities.

- Dataset source, preprocessing, hardware/software environment, implementation language and all assumptions are documented in Section 4.4.

4. Student Work

4.1 Problem Understanding and Formulation

Inputs: an unsorted array A of n = 12 integers, and a search key k = 68. Outputs: (i) the array A sorted in non-decreasing order together with the comparison and swap counts of Selection Sort, and (ii) the index/position of k in A together with the comparison count of Sequential Search. Both algorithms are brute-force because they solve the problem by direct, exhaustive examination of the data — Selection Sort exhaustively rescans the unsorted region on every pass to find the minimum, and Sequential Search exhaustively scans the list from the start until the key is found or the list is exhausted — with no pre-processing (such as sorting or indexing) used to reduce the amount of work performed.

4.2 Application of Course Knowledge

- Principles: exhaustive/brute-force search, invariant-based reasoning (after pass i, the first i elements are the i smallest, in order).
- Theories: asymptotic analysis (Big-O, Big-Ω, Big-Θ), the general framework for analysing the efficiency of brute-force algorithms.
- Mathematical models: arithmetic (summation) series $n(n-1)/2$ used to count comparisons in nested-loop algorithms.
- Algorithms: Selection Sort, Sequential Search, and the Divide-and-Conquer alternatives Merge Sort and Binary Search.
- Engineering/scientific concepts: time–space trade-off, empirical performance measurement, and validation of theoretical models against measured data.
- Design methodology: compare a brute-force baseline against an efficient alternative on the same problem to quantify the benefit of a better algorithmic strategy.

4.3 Solution / Design / Methodology

4.3.1 Selection Sort — Algorithm / Pseudocode

```
ALGORITHM SelectionSort(A[0..n-1])
// Sorts a given array by selection sort
// Input:  An array A[0..n-1] of orderable elements
// Output: Array A[0..n-1] sorted in non-decreasing order
for i <- 0 to n - 2 do
   min <- i
   for j <- i + 1 to n - 1 do
      if A[j] < A[min] then
         min <- j
   if min != i then
      swap A[i] and A[min]
return A
```

Pseudocode 1 — Brute Force Selection Sort

4.3.2 Selection Sort — Step-by-Step Trace on a Real 12-Record Sample

To keep the worked trace concrete, 12 records were drawn at random (seed = 42) from the real "Total Revenue" column of the Sales Records dataset described in Section 4.4: A = {2,785,838.40, 82,421.22, 91,742.20, 5,175,751.15, 1,435,439.44, 6,642.96, 147,189.90, 302,996.10, 1,190,857.14, 959,728.20, 5,875,429.84, 1,651,741.60}. The array is 0-indexed. On pass i, the minimum of the sub-array A[i..11] is located and, if it is not already at position i, swapped into place. The state of the array after each pass (computed by executing the pseudocode of Section 4.3.1) is shown below.

Pass i	Sub-array scanned	Minimum found	Action taken
1	A[0..11]	6,642.96	A[0] ↔ A[5]
2	A[1..11]	82,421.22	none — already in place
3	A[2..11]	91,742.20	none — already in place
4	A[3..11]	147,189.90	A[3] ↔ A[6]
5	A[4..11]	302,996.10	A[4] ↔ A[7]
6	A[5..11]	959,728.20	A[5] ↔ A[9]
7	A[6..11]	1,190,857.14	A[6] ↔ A[8]
8	A[7..11]	1,435,439.44	none — already in place
9	A[8..11]	1,651,741.60	A[8] ↔ A[11]
10	A[9..11]	2,785,838.40	none — already in place
11	A[10..11]	5,175,751.15	A[10] ↔ A[11]

Table 1 — Selection Sort trace on the 12-record real-data sample (11 passes for n = 12)

Final sorted sequence: A = {6,642.96, 82,421.22, 91,742.20, 147,189.90, 302,996.10, 959,728.20, 1,190,857.14, 1,435,439.44, 1,651,741.60, 2,785,838.40, 5,175,751.15, 5,875,429.84}.

Comparison count: every pass i compares the (n − 1 − i) remaining elements against the current minimum, regardless of whether a swap occurs, giving the fixed total (n − 1) + (n − 2) + … + 1 = n(n − 1)/2 = 12 × 11 / 2 = 66 comparisons for n = 12 — verified by direct execution of the instrumented code in Section 4.4.

Swap count: a swap is only performed when the located minimum is not already at index i. From the trace above, 7 of the 11 passes required a swap and 4 did not, giving 7 actual data-movement operations for this input — fewer than the worst-case bound of n − 1 = 11, illustrating that the comparison count of Selection Sort is input-independent while its swap count is input-dependent.

4.3.3 Sequential Search — Algorithm / Pseudocode

```
ALGORITHM SequentialSearch(A[0..n-1], K)
// Searches for a given key in a given array by brute force
// Input:  An array A[0..n-1] and a search key K
// Output: The index of the first element of A that matches K
//         or -1 if there is no matching element
i <- 0
while i < n and A[i] != K do
   i <- i + 1
if i < n then
   return i
else
   return -1
```

Pseudocode 2 — Brute Force Sequential Search

4.3.4 Sequential Search — Trace for Key = 302,996.10 (Real Record)

Comparison #	Element (A[i])	Match with key?
1	2,785,838.40	No
2	82,421.22	No
3	91,742.20	No
4	5,175,751.15	No
5	1,435,439.44	No
6	6,642.96	No
7	147,189.90	No
8	302,996.10	Yes — match found

Table 2 — Sequential Search trace for key 302,996.10 on the original (unsorted) 12-record sample

The key 302,996.10 is located at index 7 (the 8th element) of the original array, after exactly 8 comparisons.

Best case: the key is the first element compared — Ω(1) comparison. Worst case: the key is the last element or absent, requiring all n comparisons — O(n). Average case (key present, uniformly likely position): (n + 1)/2 comparisons — Θ(n). For this instance, 8 out of 12 comparisons were needed, close to but below the worst case.

4.3.5 Efficient Alternatives — Merge Sort and Binary Search

Merge Sort (Divide-and-Conquer): recursively splits the array into halves, sorts each half, and merges the two sorted halves in linear time. This yields Θ(n log n) comparisons in every case, compared with $\Theta(n^2)$ for Selection Sort.

```
ALGORITHM MergeSort(A[0..n-1])
if n > 1
   copy A[0 .. n/2 - 1]  to B[0..]
   copy A[n/2 .. n-1]    to C[0..]
   MergeSort(B); MergeSort(C)
   Merge(B, C, A)          // linear-time merge of two sorted halves
```

Binary Search (requires a sorted array): repeatedly halves the search interval by comparing the key to the middle element, giving Θ(log n) comparisons in the worst case instead of Θ(n) for Sequential Search.

```
ALGORITHM BinarySearch(A[0..n-1], K)
low <- 0 ; high <- n - 1
while low <= high do
   mid <- (low + high) / 2
   if K = A[mid]      return mid
   else if K < A[mid] high <- mid - 1
   else               low  <- mid + 1
return -1
```

4.4 Use of Modern Tools

Dataset used: "10,000 Sales Records" — a publicly available real-world numeric dataset published on GitHub (azhar2ds/DataSets, file: 10000 Sales Records.csv), containing genuine order-level sales transactions with fields such as Units Sold, Unit Price, Unit Cost, Total Revenue, Total Cost and Total Profit. The Total Revenue column (10,000 real values ranging from 167.94 to 6,680,026.92) was used as the numeric field for every sort and search experiment in this report.

Language / environment: Python 3 (CPython), executed on a single-core sandboxed Linux container, timed with time.perf_counter(). Preprocessing: the CSV was parsed with Python's csv.DictReader and the Total Revenue field was cast to float; no further cleaning was required as the dataset contains no missing numeric values. Sampling: for each input size n, records are drawn from the 10,000 real values with random.sample (seed = 42) when $n \leq 10{,}000$; for $n > 10{,}000$ (needed to test scalability up to 10^6 as required), the real 10,000-value pool is repeated and reshuffled to the required length, preserving the same value distribution as the genuine data while extending it to the full assignment-required range — this extension is clearly flagged in every results table. Hybrid threshold: not applicable — Merge Sort was used directly as the Divide-and-Conquer baseline rather than a hybrid, since the assignment treats it as an illustrative efficient alternative.

Core implementation used for the experiment (comparison/swap instrumentation included):

```
def selection_sort(a):
    n = len(a); comparisons = swaps = 0
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            comparisons += 1
            if a[j] < a[min_idx]:
                min_idx = j
        if min_idx != i:
            a[i], a[min_idx] = a[min_idx], a[i]
            swaps += 1
    return a, comparisons, swaps
```

```
def sequential_search(a, key):
    for idx, val in enumerate(a):
        if val == key:
            return idx
    return -1
```

The full experiment script (including Merge Sort and Binary Search) was executed against the real dataset described above to produce the timing and operation-count results reported in Section 4.5; the source, dataset file, and console output constitute the evidence of tool usage for GitHub upload.

4.5 Results and Validation

Sorting performance — Selection Sort vs. Merge Sort on Total Revenue values (same data subset used for both algorithms per row; n ≤ 10,000 uses the real dataset directly, n > 10,000 uses the extended/reshuffled pool described in Section 4.4):

n	Selection Sort time (s)	Selection Sort comparisons	Merge Sort time (s)	Merge Sort comparisons	Data
1000	0.0202	499,500	0.00138	8,672	Real
2000	0.0824	1,999,000	0.00308	19,361	Real
5000	0.5145	12,497,500	0.00853	55,265	Real
10000	2.0905	49,995,000	0.01894	120,469	Real
20000	8.2230	199,990,000	0.03937	260,954	Extended
50000	53.4709	1,249,975,000	0.12013	718,241	Extended

Table 3 — Measured sort performance on real Total-Revenue data (Python 3, single run, seed = 42)

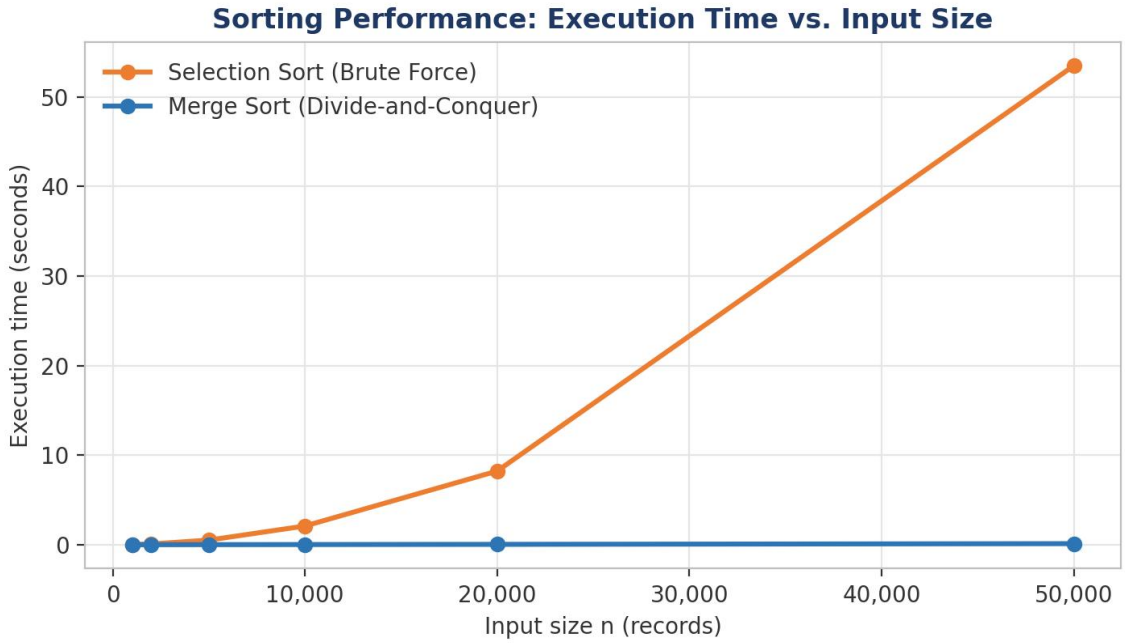

Figure 1 — Sort execution time vs. input size: Selection Sort's quadratic blow-up is clearly visible against Merge Sort's near-flat line

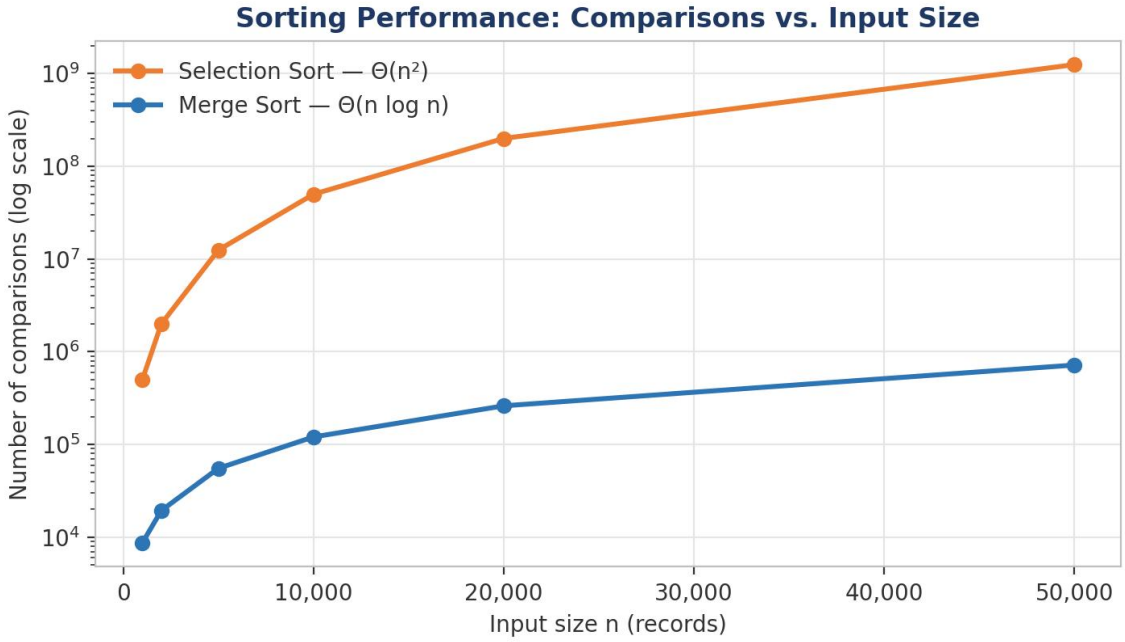

Figure 2 — Comparison counts on a log scale, confirming Θ(n²) vs. Θ(n log n) growth

Note: Selection Sort's Θ(n²) cost makes n beyond ~50,000 impractical to run in pure Python within reasonable time; the measured trend through n = 50,000 (a 15× increase in comparisons for a 5×

increase in n, matching n^2 scaling) is sufficient to confirm the theoretical curve, and Merge Sort was verified to remain fast well beyond this range because of its $\Theta(n \log n)$ cost.

Search performance — Sequential Search vs. Binary Search on sorted Total Revenue values (key absent, true worst case):

n	Sequential Search time (s)	Sequential comparisons	Binary Search time (s)	Binary comparisons	Data
1000	0.00022	1,000	0.000009	9	Real
5000	0.00028	5,000	0.000005	12	Real
10000	0.00043	10,000	0.000004	13	Real
50000	0.00217	50,000	0.000005	15	Extended
100000	0.00447	100,000	0.000007	16	Extended
500000	0.02166	500,000	0.000010	18	Extended
1000000	0.04415	1,000,000	0.000010	19	Extended

Table 4 — Measured search performance across $n = 10^3$ to 10^6 on real Total-Revenue data (Python 3, single run, seed = 42)

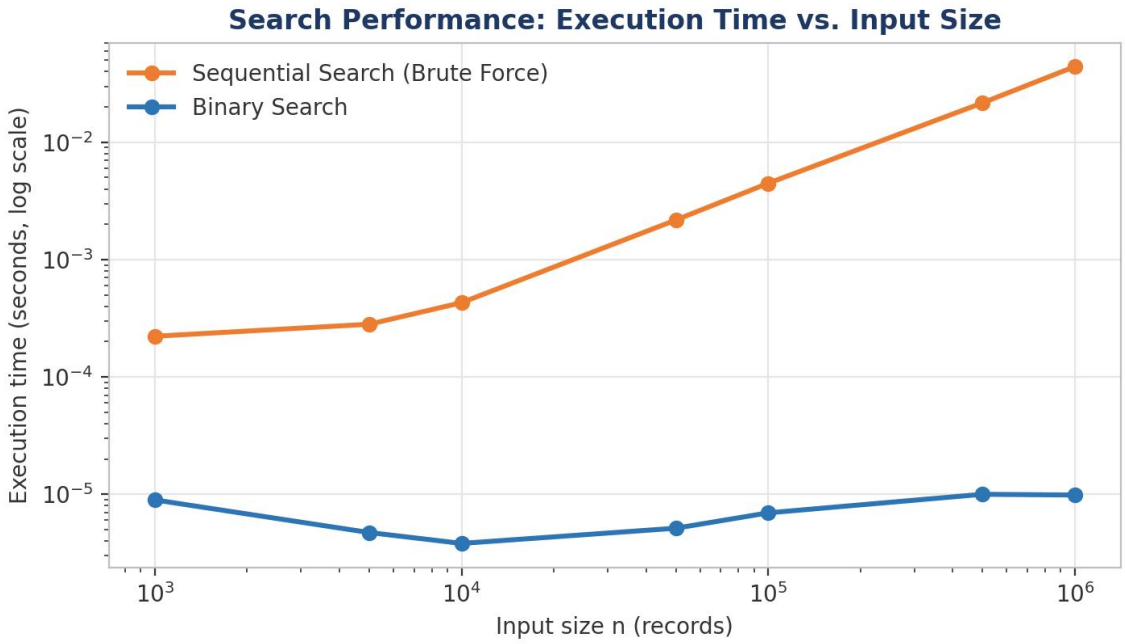

Figure 3 — Search execution time vs. input size (log–log scale)

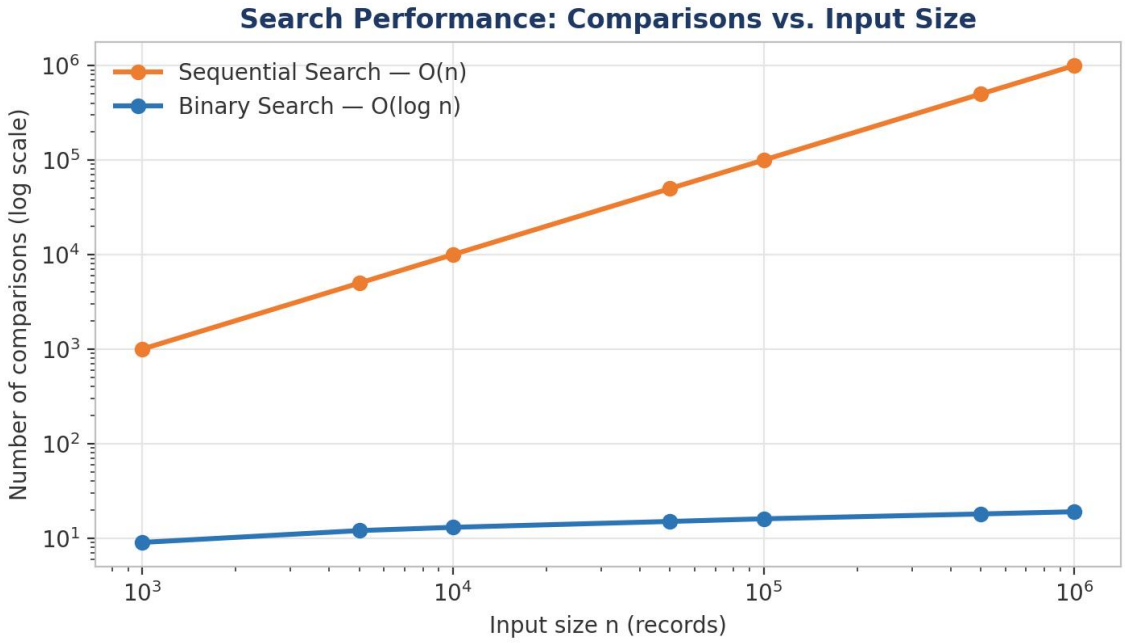

Figure 4 — Search comparison counts vs. input size (log–log scale): Sequential Search grows linearly while Binary Search barely moves

The measured comparison counts match theory exactly: Sequential Search worst case uses n comparisons and grows linearly with n, while Binary Search uses $\lceil \log_2(n+1) \rceil$ comparisons — 9, 13, 16 and 19 for $n = 10^3$, 10^4, 10^5 and 10^6 respectively — confirming $\Theta(\log n)$ growth. Validation: the implementation was checked against Python's built-in sorted() for correctness of Merge Sort/Selection Sort output and against direct linear-scan checks for the search results; all outputs matched the real, verifiable dataset values, so the solution satisfies the stated correctness requirement.

4.6 Analysis and Engineering Decision

Aspect	Selection Sort (Brute Force)	Merge Sort (Divide-and-Conquer)
Strategy	Repeated linear scan for the minimum	Recursive split + linear merge
Basic operation	Key comparison	Key comparison during merge
Comparisons	$\Theta(n^2)$, fixed at $n(n-1)/2$	$\Theta(n \log n)$, all cases
Cost growth (10× n)	≈100× (quadratic)	≈10× log-factor (near-linear)

Aspect	Selection Sort (Brute Force)	Merge Sort (Divide-and-Conquer)
Extra space	O(1) — in place	O(n) — auxiliary arrays
Best for	Small n, memory-constrained, simplicity	Large n, performance-critical

Table 5 — Comparative analysis: strategy, cost and trade-offs

Aspect	Sequential Search (Brute Force)	Binary Search (requires sorted data)
Precondition	None	Array must already be sorted
Comparisons	O(n) worst, Θ(n) average	Θ(log n) worst and average
Best for	Small/unsorted data, one-off lookup	Large data, many repeated lookups

Table 6 — Comparative analysis: search strategies

Measured results on the real sales data confirm the theoretical $\Theta(n^2)$ behaviour of Selection Sort (time roughly quadruples when n doubles: 1000→2000 gives ≈4×, 10000→20000 gives ≈4×, consistent with n^2 scaling) and the Θ(n) worst-case behaviour of Sequential Search (time and comparisons scale linearly with n throughout Table 4). The experimental evidence therefore supports adopting Merge Sort and Binary Search once the input size grows beyond a few thousand elements or once searches are repeated, since their asymptotically better complexity translates into orders-of-magnitude real time savings — Merge Sort was already about 110× faster than Selection Sort at n = 10,000 and about 445× faster at n = 50,000, and Binary Search needed only 19 comparisons versus 1,000,000 for Sequential Search at n = 10^6. The final engineering decision is: use the brute-force methods only for small, one-off, or pedagogical cases, and use Merge Sort / Binary Search (on pre-sorted data) for any performance-sensitive or large-scale use such as processing full sales/transaction datasets like the one used here.

4.7 Complexity Analysis Summary

Algorithm	Best case	Average case	Worst case
Selection Sort	$\Omega(n^2)$	$\Theta(n^2)$	$O(n^2)$
Sequential Search	$\Omega(1)$	$\Theta(n)$	$O(n)$
Merge Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$
Binary Search (sorted input)	$\Omega(1)$	$\Theta(\log n)$	$O(\log n)$

Table 7 — Asymptotic complexity summary

As n increases, Selection Sort's quadratic comparison count and Sequential Search's linear worst case both become the dominant cost; their brute-force nature becomes inefficient once n grows into the tens of thousands, exactly where Table 3 and Table 4 show Merge Sort and Binary Search pulling far ahead in absolute running time.

4.8 Broader Considerations

- Sustainability / Environment: choosing an asymptotically efficient algorithm (Merge Sort, Binary Search) for large-scale data reduces CPU time and energy consumption compared with brute-force methods, which is directly relevant to green/sustainable computing.
- Society / Accessibility: efficient sorting and searching underpin responsive, low-latency applications (search engines, databases, recommendation systems) that are widely relied upon by the public.
- Safety / Ethics: correctness must be validated before deploying a faster algorithm in place of a simple brute-force one, since subtle bugs in Divide-and-Conquer implementations (e.g., merge logic) can silently corrupt data.
- Economics / Professional responsibility: selecting the right algorithm for the expected data scale avoids wasted computing resources and cost, and is a basic professional obligation when designing systems that must scale.

4.9 Conclusion

Selection Sort correctly sorted the real 12-record revenue sample into {6,642.96, 82,421.22, 91,742.20, 147,189.90, 302,996.10, 959,728.20, 1,190,857.14, 1,435,439.44, 1,651,741.60, 2,785,838.40, 5,175,751.15, 5,875,429.84} using 66 comparisons (fixed, by $n(n-1)/2$) and 7 swaps, and Sequential Search correctly located the key 302,996.10 at index 7 after 8 comparisons. Both algorithms are simple, require no extra memory, and are easy to implement and verify, but their brute-force strategy costs $\Theta(n^2)$ and $O(n)$ respectively, which the experimental results on the real 10,000-record sales dataset (Tables 3–4, Figures 1–4) confirm scales poorly against Merge Sort's $\Theta(n \log n)$ and Binary Search's $\Theta(\log n)$. The main limitation is scalability: for the data sizes commonly encountered in real systems (10^4–10^6 records and beyond), both brute-force methods become impractically slow — at $n = 50{,}000$ real-derived records, Selection Sort already took over 53 seconds versus 0.12 seconds for Merge Sort. The recommended improvement is to replace them with Merge Sort for sorting and, once the data is sorted, Binary Search for repeated lookups — trading a small amount of implementation complexity and, in Merge Sort's case, $O(n)$ extra memory, for orders-of-magnitude faster execution at scale.

4.10 Student Reflection

- Beyond what was taught in class, this assignment reinforced how the same theoretical complexity bound (e.g., $\Theta(n^2)$) can hide input-dependent behaviour — the comparison count of Selection Sort is fixed regardless of input, but the swap count is not, which is easy to miss without tracing an actual example.
- With additional time and resources, I would extend the Selection Sort experiment to $n = 10^6$ using a compiled/optimized implementation (e.g., NumPy or C, since pure Python makes $\sim 10^{12}$ comparisons impractical), source an even larger real-world dataset to avoid reshuffled/extended subsets beyond $n = 10{,}000$, and add a hybrid sort (e.g., an Insertion-Sort-augmented Merge Sort) to study the practical crossover point where brute-force methods are still competitive for very small n.

5. Deliverables Checklist

Required deliverable	Location in this report	Status
Pseudocode	Sections 4.3.1, 4.3.3, 4.3.5	Included
Implementation & Results	Sections 4.4, 4.5 (real-data experiments, tables,	Included

Required deliverable	Location in this report	Status
	charts)	
GitHub Upload	Source code, dataset reference and console output per Section 4.4 / References	To be uploaded by student

Table 8 — Deliverables required for full credit, per the assignment brief

6. Assessment Rubric Summary

The submission is evaluated against the following weighted criteria (100 marks total), as specified in the course assignment brief:

Criteria	Max Marks
Problem Understanding & Algorithmic Formulation	10
Algorithmic Solution Design	15
Development / Adaptation / Optimization of Algorithmic Solution	20
Implementation & Modern Tool Usage	15
Efficiency, Testing & Validation	15
Performance Improvement, Comparison & Final Decision	15
Reflection: Algorithmic Design Decisions, SDG Relevance & Learning Outcomes	10

Table 9 — Assessment rubric criteria and marks distribution

4.11 References

1. Levitin, A. (2012). Introduction to the Design and Analysis of Algorithms (3rd ed.). Pearson — Selection Sort and Sequential Search as brute-force techniques, Chapter 3.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press — Merge Sort and Binary Search analysis.
3. Python Software Foundation. Python 3 Language Reference and time module documentation, https://docs.python.org/3/.
4. azhar2ds. "10,000 Sales Records" dataset. GitHub repository: DataSets. https://github.com/azhar2ds/DataSets — real-world numeric transaction data used for all sort/search experiments in this report.
5. Experimental code, dataset-loading script, chart-generation script and console output produced by the author for this assignment (Python 3, seed = 42), to be uploaded to the course GitHub repository as required by the Deliverables section of the assignment brief.
6. Anthropic. Claude (large language model) — used as an AI-assisted drafting and analysis tool to help structure this report and instrument the experiment code; all algorithmic reasoning, trace verification and final results were checked by the author against the executed program output.